```
iex> to_bool("undefined") # false
iex> to_bool("UNDEFINED") # false
iex> to_bool(false) # false

iex> to_bool(true) # true
iex> to_bool("A string") # true
iex> to_bool(1) # true
```

Even though we've defined to_bool/1 over five times, everything still works as expected. A mix of pattern matching and guards pulls this off for us.

An example of guard limitations is seen on line 10. We would ideally check the input value in a case-insensitive way, but there's no way to do this in the guard. A solution to this is seen on line 12. The input is guarded as a string, and then the function body performs the downcasing.

The final trick is seen on line 16. This function accepts any value, so to_bool/1 is guaranteed to always have a matching function. If you remove this line, you'll see a FunctionClauseError for inputs like to_bool(1).

The Secret to Function Overloading

There's no magic in Elixir's function overloading—it's entirely based on things you already know! And once you know how it works, you'll better understand why functions are evaluated from top to bottom.

A function with multiple heads gets converted to a single function consisting of a case statement. A more basic to_bool/1 function might look like this:

elixir_examples/lib/examples/patterns/boolean_case.ex

```
defmodule Examples.Patterns.BooleanCase do
  def to_bool(value) do
    case value do
      bool when is_boolean(bool) -> bool
      0 -> false
      nil -> false
      str when str in ["undefined", "false"] -> false
      _ -> true
    end
  end
end
```

This is all handled by the Elixir compiler, and it completely explains the top-to-bottom definition order. If we define a function that accepts any value at the top of our file, then it will be first in the case statement and will always be triggered. This also shows that guards work in case statements because they're converted to case statements by Elixir's compiler.